PADHAANEWALA EDUTECH SERVICES

BENGALURU – 560100

MASTER WEBSITE DEVELOPMENT SPECIFICATION

Assigned Role: Website Engineering & Product Development Team

Project: Padhaanewala Education Technology Platform
Website: padhaanewala.in
Document: Complete Product, Technical & Development Requirements
Version: 1.0

1. PROJECT OBJECTIVE

Develop Padhaanewala as a professional, scalable, database-driven education platform.

The website must NOT be developed as a collection of static pages.

It must be a complete platform consisting of: Student-facing website; College database; Course database; Scholarship database; Examination database; Mock-test system; AI College Predictor; AI Education Assistant; College comparison; Student accounts; Reviews; Admission enquiry system; Counsellor/lead management; Admin dashboard; Content Management System; SEO infrastructure; Analytics.

The architecture must allow the platform to expand to a large number of colleges, courses and users without requiring a complete rewrite.

2. CORE BUSINESS PURPOSE

Padhaanewala should help students: discover colleges; search courses; compare colleges; understand eligibility; check fees; understand admission procedures; find scholarships; find examination information; take mock tests; use an AI college predictor; ask education-related questions; save colleges; submit admission enquiries; receive counselling assistance.

The platform should simultaneously generate and manage admission leads for Padhaanewala.

3. TECHNOLOGY STACK

Frontend: Next.js, React, TypeScript, Tailwind CSS, responsive/mobile-first design. Next.js is preferred over a basic React SPA because SEO is extremely important for college/course pages.

Backend: Python, FastAPI, REST API, API versioning.

Database: PostgreSQL.

Cache: Redis where required.

Search: PostgreSQL search initially; Elasticsearch/OpenSearch when scale requires it.

Storage: AWS S3, Cloudflare R2 or equivalent S3-compatible object storage. Do not store large images directly in PostgreSQL.

AI: AI requests must go through the backend. Never expose AI API keys in frontend code.

4. DOMAIN AND DEVELOPMENT ENVIRONMENT

Development may use localhost and staging before the production domain is configured.

Production domain: padhaanewala.in

Maintain separate Development, Staging and Production environments.

Never test major experimental changes directly on production.

5. WEBSITE NAVIGATION

Main navigation: Home, Colleges, Courses, College Predictor, Scholarships, Mock Tests, Exams, Reviews, Blog/Resources, About, Contact.

Also provide Login, Register and Get Admission Help.

6. HOMEPAGE

The homepage must look like a modern education technology platform and NOT like a generic coaching-centre website.

Hero: “Find the Right College for Your Future”.

Provide a large search bar: “Search colleges, courses, exams or locations”. Examples: BHMS, BAMS, BUMS, MBBS, BDS, B.Sc Nursing, B.Pharm, D.Pharm, BCA, MBA, Engineering.

Buttons: Search; AI College Predictor.

Quick-action cards: Find Colleges; Compare Colleges; College Predictor; Scholarships; Mock Tests; Admission Assistance.

Homepage sections: Popular courses; Featured colleges; Popular college searches; Scholarships; Upcoming examinations; Mock tests; Why Padhaanewala; Student reviews; Latest education articles; Admission assistance CTA.

All homepage content must be manageable from the admin panel.

7. COLLEGE DATABASE

Create a structured college database. Every college must have a unique ID, e.g. COLLEGE000001.

Fields: College ID; College name; Official name; College type; Government/private; University; State; District; City; Address; Pincode; Website; Email; Phone; Established year; Accreditation; Recognition; Courses; Fees; Hostel; Facilities; Admission information; Eligibility; Entrance examination; Cutoff; Images; Gallery; Reviews; FAQs; Data source; Last verified date; Verification status.

8. COLLEGE PAGE

Every college must have an individual SEO-friendly page, e.g. /college/college-name.

Header: College name; Location; College type; Rating/reviews; Apply/Get Admission Help; Compare; Save College.

Sections: Overview; Courses; Fees; Eligibility; Admission; Cutoff; Facilities; Reviews; Gallery; FAQs.

9. COURSE DATABASE

Courses must be separate database entities.

Examples: MBBS; BDS; BAMS; BHMS; BUMS; B.Sc Nursing; D.Pharm; B.Pharm; BCA; BBA; MBA; Engineering; Paramedical courses; Skill courses.

Each course should have: Name; Degree; Duration; Eligibility; Entrance examination; Admission procedure; Fees information; Colleges offering the course; Career information; FAQs; SEO metadata.

10. COURSE PAGE

Example: /courses/bhms

Sections: Overview; Duration; Eligibility; Admission; Entrance exam; Fees; Colleges; Career opportunities; FAQs; Related courses.

CTA: “Need help choosing a college?” → Get Admission Assistance.

11. COLLEGE SEARCH

Create advanced college search.

Filters: Course; State; District; City; Government/private; University; Fees; Hostel; Rating; Accreditation; Admission status.

Example: BHMS + Karnataka + Private returns matching colleges.

12. NATURAL LANGUAGE SEARCH

Allow users to search naturally, for example: “BHMS colleges in Karnataka”; “BAMS colleges near Bangalore”; “Nursing colleges under ■5 lakh”; “Private BHMS colleges with hostel”.

The backend should convert natural-language requests into appropriate structured search filters wherever possible.

13. COLLEGE COMPARISON

Students should be able to select multiple colleges.

Comparison should show: Location; Type; University; Course; Duration; Fees; Hostel; Facilities; Admission; Reviews; other relevant information.

Include: “Ask AI: Which college is better for me?” AI recommendations must be based on available verified database information and clearly state that recommendations are not admission guarantees.

14. AI COLLEGE PREDICTOR

This should be one of the main Padhaanewala features.

Inputs: Course; Entrance exam; Rank/score; Category where relevant; State; Preferred city; Budget; Government/private preference; Hostel requirement; Other preferences.

Output: Highly Suitable; Possible; Reach.

The predictor must clearly state that results are estimates and not guaranteed admissions.

15. AI ARCHITECTURE

Recommended flow: Student → Frontend → FastAPI → Database/Search Engine → Relevant verified data → AI processing → Response.

Do NOT allow the AI model to independently invent factual college information.

The AI should use database information for factual information such as College names, Courses, Fees, Locations, Eligibility and Admission information.

Where current information is uncertain, the system should indicate that the information needs verification.

16. AI EDUCATION ASSISTANT

Add: “Ask Padhaanewala AI”.

Possible questions: What is BHMS? What is the difference between BAMS and BHMS? Which course is suitable after 12th? Which colleges offer B.Sc Nursing? What scholarships are available? How does admission work?

The system should not present changing regulatory/admission information as permanently fixed facts.

17. SCHOLARSHIP FINDER

Create a separate scholarship database.

Fields: Scholarship name; Provider; Government/private; Eligibility; State; Course; Income criteria; Amount; Deadline; Documents; Application procedure; Official application source; Status; Last verified date.

Filters: Course; State; Student category where applicable; Income; Government/private; Deadline.

18. SCHOLARSHIP PAGE

Each scholarship should have: Scholarship name; Provider; Amount; Eligibility; Deadline; Required documents; Application process; Official application link; FAQs.

Clearly distinguish Official scholarship application from Padhaanewala counselling/admission assistance.

19. EXAMINATION DATABASE

Create /exams.

Each exam should include: Exam name; Conducting authority; Eligibility; Application start date; Application deadline; Exam date; Admit card date; Result date; Official website; Official notification; FAQs.

All dates must be editable through the admin panel.

20. MOCK TEST SYSTEM

Create /mock-tests.

Features: Exam selection; Subject selection; Difficulty; Number of questions; Timer; Question navigation; Mark for review; Next/previous; Submit.

21. MOCK TEST RESULT

After submission show: Score; Percentage; Correct answers; Incorrect answers; Unattempted; Time taken; Topic-wise performance; Rank/percentile where meaningful.

Buttons: Practice Again; View Solutions.

22. STUDENT REGISTRATION

Students should be able to register using Mobile OTP and/or Email/Password.

Do not store passwords as plain text. Use secure password hashing.

23. STUDENT PROFILE

Profile should contain: Name; Mobile; Email; Education; Course interest; Preferred state; Preferred city; Budget; Saved colleges; Test history; Scholarship interests; Enquiries.

24. SAVE COLLEGE

Students should be able to click Save College.

Saved colleges should appear under My Colleges.

Students should then be able to compare them.

25. ADMISSION ENQUIRY

Every important page should have Get Admission Assistance.

Form: Name; Mobile; Email; Course; Preferred college; State; City; Qualification; Message.

After submission: “Thank you. Our counsellor will contact you.”

The enquiry must immediately enter the admin/CRM system.

26. LEAD MANAGEMENT

Each lead should have: Lead ID; Student name; Mobile; Email; Course; College; Source; Date; Status; Assigned counsellor; Follow-up date; Notes.

Statuses: New; Contacted; Interested; Application Started; Admission Completed; Not Interested; Closed.

27. COUNSELLOR DASHBOARD

Counsellors should have their own login.

Features: Assigned leads; Student details; Notes; Follow-up; Contact status; Admission status.

Use role-based access so counsellors cannot access information they are not authorized to see.

28. ADMIN PANEL

Admin must be able to manage the entire platform.

Modules: Dashboard; Colleges; Courses; Scholarships; Exams; Mock Tests; Questions; Students; Reviews; Blogs; FAQs; Banners; Notifications; Leads; Counsellors; Media; SEO; Settings; Audit logs.

29. ADMIN ROLES

Super Admin: Full access.

Content Admin: College/course/blog/scholarship content.

Counsellor: Assigned leads.

Test Admin: Mock tests/questions.

SEO Admin: SEO metadata/content.

Permissions must be configurable.

30. CMS

Non-technical administrators must be able to update website content.

Admin should be able to Add, Edit, Delete, Publish, Unpublish and Schedule content.

This should include Colleges, Courses, Scholarships, Exams, Blogs, FAQs, Homepage content and Banners.

31. BLOG

Create /blog.

Categories: Admissions; NEET; AYUSH; Nursing; Scholarships; Careers; Exams; College guides; Education news.

Each article needs: Title; Slug; Content; Featured image; Category; Author; Meta title; Meta description; Canonical URL; Publish status.

32. SEO

Every important page must have: SEO title; Meta description; Canonical URL; Open Graph metadata; Structured data; Sitemap; Robots.txt; Clean URL.

Example: /colleges/bhms-colleges-in-karnataka is preferable to /page?id=123.

33. PROGRAMMATIC SEO

The platform may automatically create useful pages such as BHMS colleges in Karnataka, BAMS colleges in Karnataka, Nursing colleges in Bihar and B.Pharm colleges in Bangalore.

Do NOT generate thousands of low-quality duplicate pages. Only useful, sufficiently informative pages should be indexable.

34. COLLEGE DATA VERIFICATION

Every important college record should contain Data source; Verification status; Last verified date; Verified by.

Possible sources: Official college website; Official government source; Regulatory authority; University; Verified institutional communication.

35. FEES DATA

Fees should be structured rather than a single text field.

Possible fields: Tuition fee; Hostel fee; Examination fee; Other charges; Total approximate fee; Fee period.

Display “Approximate fee” when exact/current fee cannot be guaranteed.

Include: “Fees should be verified with the institution before admission.”

36. REVIEWS

Students can submit: College; Course; Year; Rating; Review; Optional images.

Reviews must go through moderation: Submitted → Moderation → Approved → Published.

Admin should be able to reject spam, abusive or inappropriate content.

37. COLLEGE GALLERY

Images should be stored using object storage.

Database should store: Image URL; College ID; Image type; Alt text; Upload date.

Images must be optimized before delivery.

38. WHATSAPP

Add WhatsApp CTA where appropriate.

Example contextual message: “Hello Padhaanewala, I am interested in BHMS admission at [College Name].”

The WhatsApp number should be configurable from the admin/settings panel rather than hard-coded throughout the frontend.

39. CONTACT PAGE

Include: Company information; Phone; Email; WhatsApp; Working hours; Contact form; Location/map where appropriate; Social links.

40. ABOUT PAGE

Explain: Padhaanewala; Mission; Vision; Services; College discovery; Student support; Counselling.

Avoid legally risky or unverifiable claims such as guaranteed admission unless they are genuinely supportable.

41. LEGAL PAGES

Create: Privacy Policy; Terms & Conditions; Disclaimer; Cookie Policy where applicable; Refund/Cancellation Policy if payments are introduced.

42. API ARCHITECTURE

Use versioned APIs such as /api/v1/auth, /users, /colleges, /courses, /scholarships, /exams, /mock-tests, /reviews, /blog, /enquiries, /predictor, /ai and /admin.

APIs must be documented, preferably through FastAPI OpenAPI/Swagger.

43. COLLEGE API

Example endpoints: GET /api/v1/colleges; GET /api/v1/colleges/{id}; POST /api/v1/colleges; PUT /api/v1/colleges/{id}; DELETE /api/v1/colleges/{id}.

Public users should only have access to appropriate public endpoints. Admin endpoints must require authentication and authorization.

44. SEARCH API

Example: GET /api/v1/colleges.

Parameters: course; state; district; city; college_type; min_fee; max_fee; rating; page; limit; sort.

Do not return thousands of records in one response. Use pagination.

45. DATABASE ENTITIES

Minimum suggested entities: users; student_profiles; admins; counsellors; colleges; college_courses; courses; universities; locations; fees; admissions; scholarships; exams; exam_dates; mock_tests; questions; answers; test_attempts; reviews; blogs; categories; enquiries; lead_notes; notifications; saved_colleges; faqs; media; seo_metadata; audit_logs.

Engineer may normalize this structure as required.

46. IMPORT SYSTEM

Admin should be able to upload college/course information using CSV/Excel.

Example columns: College Name; State; District; City; University; Course; Fees; Website; Phone; Email; Address.

Process: Upload → Validate → Show Errors → Preview → Confirm → Import.

47. DUPLICATE DETECTION

The import system should detect possible duplicate colleges.

Do not create duplicate records simply because names differ slightly. Similar institutional names must be checked before creating separate records.

48. ANALYTICS

Website: Visitors; Searches; College views; Course views; Predictor usage.

Students: Registrations; Active users.

Leads: New leads; Contacted; Converted.

Content: Most viewed colleges; Most searched courses; Popular scholarships; Popular articles.

49. LEAD SOURCE TRACKING

Store source for each enquiry.

Examples: Homepage; College page; Course page; Blog; Scholarship; Predictor; WhatsApp; Advertisement; Organic search.

This is important for measuring marketing performance.

50. UTM TRACKING

Support utm_source, utm_medium, utm_campaign and utm_content.

Where practical, associate campaign information with the lead.

51. GOOGLE SERVICES

Configure Google Analytics, Google Search Console and Google Tag Manager if needed.

Track searches, college views, predictor usage, enquiries, registrations, WhatsApp clicks, phone clicks and scholarship clicks.

52. SECURITY

Mandatory security requirements: HTTPS; secure authentication; password hashing; secure sessions/JWT; input validation; SQL injection protection; XSS protection; rate limiting; API authorization; file-upload validation; secure HTTP headers; admin 2FA where possible; audit logs.

Never store passwords in plain text or expose API keys in frontend code.

53. ENVIRONMENT VARIABLES

Use secure environment variables.

Examples: DATABASE_URL; JWT_SECRET; AI_API_KEY; EMAIL_API_KEY; STORAGE_ACCESS_KEY; STORAGE_SECRET.

Create separate configurations for development, staging and production.

54. PERFORMANCE

Optimize with server-side rendering/static generation where appropriate, image optimization, lazy loading, code splitting, CDN, API caching, database indexing and Redis caching.

Avoid unnecessary animations and oversized images.

55. MOBILE RESPONSIVENESS

The website must work properly on Android, iPhone, tablets, laptops and desktop.

Mobile should not simply be a scaled-down desktop interface. Important actions must be easy to use with touch.

56. ACCESSIBILITY

Implement proper heading hierarchy, alt text, accessible buttons, form labels, keyboard navigation, good contrast and screen-reader-friendly structure.

57. ERROR HANDLING

Do not expose technical errors to users.

Instead of showing a database or server error, show: “Something went wrong. Please try again.”

Technical details should be stored in server logs.

58. LOGGING AND MONITORING

Maintain application logs for API errors, authentication failures, admin changes, AI failures, database errors and payment errors if introduced.

Set up monitoring/alerts for critical production failures.

59. BACKUPS

Database backup must be automated.

Recommended: Daily backups; retention policy; off-site backup; periodic restoration testing.

A backup must be tested to ensure it can actually be restored.

60. GIT AND VERSION CONTROL

Use Git.

Suggested structure: main; develop; feature/*; bugfix/*.

Use pull requests/code review where practical.

Never commit production secrets.

61. TESTING

Frontend: Test forms, navigation, search, mobile layouts, authentication and college pages.

Backend: Test APIs, authentication, authorization, database operations, search, predictor and enquiries.

End-to-end: Student → Search → College → Enquiry; Student → Predictor → Results → Enquiry; Student → Mock Test → Submit → Result; Student → Scholarship → Official application information; Admin → Add College → Publish → Public college page.

62. ADMIN CONTENT WORKFLOW

For important information use: Draft → Review → Publish.

This is especially useful for Fees, Admission dates, Scholarships and Examination dates.

63. PHASED DEVELOPMENT

Phase 1 — Core Platform: Homepage; College database; College search; College pages; Course pages; Admin panel; CMS; Enquiry system; Authentication; Basic SEO.

Phase 2 — Student Features: Student dashboard; Saved colleges; College comparison; Scholarship finder; Exam notifications; Reviews.

Phase 3 — AI: AI College Predictor; AI Education Assistant; AI college comparison.

Phase 4 — Mock Tests: Question bank; Test engine; Timer; Results; Performance analytics.

Phase 5 — Business/CRM: Counsellor dashboard; Lead assignment; Follow-ups; CRM; Marketing attribution; Advanced analytics; Automated communication.

64. FINAL SYSTEM ARCHITECTURE

Recommended architecture:

Students/Admin → Next.js Frontend/Admin Dashboard → REST APIs → FastAPI → PostgreSQL / Redis / AI / Storage / Search.

Database contains Colleges, Courses, Scholarships, Exams, Students, Reviews, Mock Tests, Leads and Content.

Architecture must be scalable and maintainable.

65. MOST IMPORTANT DEVELOPMENT RULES

1. Do not hard-code college/course/fee/scholarship data into React components.

2. All major content must be database-driven.

3. Admin must be able to update content without modifying source code.

4. Frontend and backend must communicate through documented APIs.

5. AI must not be allowed to freely invent factual college information.

6. Security must be implemented from the beginning, not added at the end.

7. Mobile responsiveness must be built from the beginning.

8. SEO must be considered during architecture, not after development.

9. All important data should have a source and/or verification date where applicable.

10. The platform must be designed so additional colleges, courses, states, users and features can be added without redesigning the entire platform.

66. DEVELOPER DELIVERABLES

At completion, the engineer/team must provide: Complete source code; Frontend code; Backend code; Database schema; API documentation; Admin credentials; Deployment documentation; Environment-variable documentation; Database backup procedure; Git repository; Testing report; Production deployment; Staging deployment; Domain/DNS configuration documentation; Third-party service documentation.

Padhaanewala should retain appropriate ownership/access to source code, hosting account, domain, database and relevant third-party accounts.

67. DEFINITION OF READY FOR LAUNCH

The website is considered ready for production only when: Homepage works; College search works; College pages work; Course pages work; Admin works; Content can be edited without code; Enquiries reach admin; Authentication works; Mobile version works; SEO basics are configured; HTTPS works; Database backup works; Security testing is completed; Major APIs are tested; Error handling works; Analytics is configured; Sitemap is available; Production deployment is stable.

68. FUTURE EXPANSION

The architecture should allow future features such as Online counselling; Paid counselling; Application tracking; Student document management; Online payments; Premium memberships; College advertising; Sponsored listings; College CRM; Partner college dashboard; Student application dashboard; Scholarship application tracking; AI career counselling;

Personalized course recommendations; Mobile application; Push notifications; Multilingual support.

These features do not necessarily need to be built in Version 1, but the architecture should not prevent them from being added later.

69. FINAL INSTRUCTION TO ENGINEER

Padhaanewala should be developed as a scalable education technology platform, not merely as a website.

Priority: Reliable data + excellent search + strong college pages + admin control + lead generation + SEO + scalable backend.

AI, mock tests and advanced features should be built on top of this foundation.

The founder/admin must be able to manage the platform without depending on the developer for routine content changes.

The final product should be fast, secure, mobile-friendly, SEO-friendly, scalable and easy to maintain.

ASSIGNED ROLE STATEMENT

As the assigned Website Engineering & Product Development Team of Padhaanewala Edutech Services, Bengaluru – 560100, the engineering team is responsible for translating this specification into a secure, scalable, maintainable and production-ready education technology platform. Any architectural deviation that materially affects scalability, SEO, security, data ownership, admin control or future expansion should be discussed and approved before implementation.

PADHAANEWALA EDUTECH SERVICES

BENGALURU – 560100

End of Master Website Development Specification